

COMPUTER SYSTEM DESIGN — LAB ASSIGNMENT

Building a Load Balancer in Go

Reverse proxy, round-robin scheduling, health checks and performance experiments — applied to the CSD Group Chat messaging backend

STUDENT NAME

Kunal Sewal

ROLL NUMBER

12341270

ASSIGNMENT TYPE

Individual

LOAD-BALANCED URL

<https://10.1.75.53:3265/>

1. Assigned Systems

All four systems share the same host, differentiated by SSH port. Each system also exposes exactly one additional application port, mapped as `external = ssh_port + 1000`, with the app itself bound to the fixed internal port `3000` for that mapping to take effect.

System	Role	SSH access	Application address
stu77_sys1	Load Balancer	<code>ssh -p 2265 student@10.1.75.53</code>	<code>https://10.1.75.53:3265/</code>
stu77_sys2	Backend replica 1	<code>ssh -p 2266 student@10.1.75.53</code>	<code>http://10.1.75.53:3266/</code>
stu77_sys3	Backend replica 2	<code>ssh -p 2267 student@10.1.75.53</code>	<code>http://10.1.75.53:3267/</code>
stu77_sys4	Backend replica 3	<code>ssh -p 2268 student@10.1.75.53</code>	<code>http://10.1.75.53:3268/</code>

2. Architecture Overview

The load balancer (Sys1) is a reverse proxy written in Go, using only the standard library (`net/http` , `net/http/httputil`). It distributes requests round-robin across whichever backends are currently healthy, retries a different backend if one errors mid-request, and terminates TLS with a self-signed certificate so the browser gets a secure context.

The backend is not a toy server — it is three independent, identically-configured copies of the actual messaging application from the previous assignment (Node.js + Express + Socket.IO + MongoDB), running on Sys2, Sys3 and Sys4. All three point at the same MongoDB Atlas cluster and the same `CHAT_ENCRYPTION_KEY`, so they read and write the same chat history and are true interchangeable replicas.

The load generator runs on the local machine and drives concurrent HTTP load at the load balancer's public address, measuring throughput, dropout rate and latency percentiles client-side.

Key load balancer features implemented

- **Round-robin scheduling** with health-awareness — an atomic cursor walks the backend list, skipping any backend currently marked unhealthy.
- **Active health checks** — a background goroutine polls each backend's `/health` endpoint every second and flips its alive/dead state.

- **Retry across backends** — if a request to the selected backend fails (dial error, response-header timeout) before any response has been sent to the client, the load balancer falls through to the next healthy backend instead of failing the request outright.
- **Reverse proxying via `httputil.ReverseProxy`**, including working WebSocket-upgrade support (needed for Socket.IO), with a custom `ResponseWriter` wrapper that still forwards `Hijack` and `Flush` correctly.
- **Configurable timeouts** for dialing and response headers, tuned separately from the timeout that would otherwise apply to a live, long-lived WebSocket connection.
- **TLS termination at the edge** — the load balancer serves HTTPS with a self-signed certificate; backend traffic on the trusted internal network stays plain HTTP.
- **Monitoring endpoints:** `/lb/health`, `/lb/status` (per-backend alive/in-flight state), `/lb/metrics` (aggregate counters and latency percentiles).

Both long-lived processes (the load balancer on Sys1 and each backend copy on Sys2–4) run inside detached `tmux` sessions, so they keep running independently of any SSH connection.

3. Load Balancer Code

`loadbalancer/main.go` — the complete reverse proxy implementation.

```
// Command loadbalancer is a round-robin, health-aware reverse proxy in front
// of one or more copies of the csd-group-chat messaging backend.
//
// It is deployed on Sys1 and forwards to backend copies of server.js running
// on Sys2, Sys3 and Sys4 (each pointed at the same MongoDB cluster and the
// same CHAT_ENCRYPTION_KEY, so they all read/write the same chat history).
package main

import (
    "bufio"
    "bytes"
    "encoding/json"
    "errors"
    "flag"
    "io"
    "log"
    "net"
    "net/http"
    "net/http/httputil"
    "net/url"
```

```

    "sort"
    "strings"
    "sync"
    "sync/atomic"
    "time"
)

// Backend is one messaging-app instance the load balancer can forward to.
type Backend struct {
    URL      *url.URL
    Alive     atomic.Bool
    InFlight  atomic.Int64
    proxy     *httputil.ReverseProxy
}

// Metrics are the load balancer's own request counters, independent of
// whatever a load generator records client-side. Exposed at /lb/metrics.
type Metrics struct {
    Total      atomic.Uint64
    Success    atomic.Uint64
    Failed     atomic.Uint64
    BackendErrors atomic.Uint64

    mu        sync.Mutex
    latencies []time.Duration
    maxSamples int
}

func (m *Metrics) record(d time.Duration) {
    m.mu.Lock()
    defer m.mu.Unlock()
    if len(m.latencies) >= m.maxSamples {
        // Drop the oldest sample so long-running processes do not grow this
        // slice without bound.
        m.latencies = m.latencies[1:]
    }
    m.latencies = append(m.latencies, d)
}

func percentile(sorted []time.Duration, p float64) time.Duration {
    if len(sorted) == 0 {
        return 0
    }
    idx := int(p * float64(len(sorted)-1))
    return sorted[idx]
}

```

```

func (m *Metrics) snapshot() map[string]any {
    m.mu.Lock()
    latencies := append([]time.Duration(nil), m.latencies...)
    m.mu.Unlock()

    sort.Slice(latencies, func(i, j int) bool { return latencies[i] < latencies[j]
})

    toMs := func(d time.Duration) float64 { return float64(d.Microseconds()) /
1000.0 }

    return map[string]any{
        "total":      m.Total.Load(),
        "success":    m.Success.Load(),
        "failed":     m.Failed.Load(),
        "backend_errors": m.BackendErrors.Load(),
        "p50_ms":     toMs(percentile(latencies, 0.50)),
        "p95_ms":     toMs(percentile(latencies, 0.95)),
        "p99_ms":     toMs(percentile(latencies, 0.99)),
    }
}

// LoadBalancer holds the backend list and the shared round-robin cursor.
type LoadBalancer struct {
    backends []*Backend
    next      atomic.Uint64
    metrics   *Metrics
}

// candidateBackends returns every currently-healthy backend, starting from
// the shared round-robin cursor and wrapping around once. Callers try them
// in this order, falling through to the next one if a request fails, so a
// single backend erroring out under load does not fail the request outright
// as long as another healthy backend exists.
func (lb *LoadBalancer) candidateBackends() []*Backend {
    n := len(lb.backends)
    if n == 0 {
        return nil
    }
    start := int(lb.next.Add(1) % uint64(n))
    candidates := make([]*Backend, 0, n)
    for i := 0; i < n; i++ {
        b := lb.backends[(start+i)%n]
        if b.Alive.Load() {
            candidates = append(candidates, b)
        }
    }
}

```

```

    }
}
return candidates
}

func (lb *LoadBalancer) ServeHTTP(w http.ResponseWriter, r *http.Request) {
    switch r.URL.Path {
    case "/lb/health":
        w.WriteHeader(http.StatusOK)
        w.Write([]byte("ok\n"))
        return
    case "/lb/status":
        lb.writeStatus(w)
        return
    case "/lb/metrics":
        w.Header().Set("Content-Type", "application/json")
        json.NewEncoder(w).Encode(lb.metrics.snapshot())
        return
    }

    lb.metrics.Total.Add(1)

    candidates := lb.candidateBackends()
    if len(candidates) == 0 {
        lb.metrics.Failed.Add(1)
        http.Error(w, "no healthy backend available",
http.StatusServiceUnavailable)
        return
    }

    // Buffer the request body once (it is a stream and can only be read a
    // single time) so it can be replayed if the first backend tried fails
    // before we've committed a response, letting us retry against the next
    // healthy candidate instead of failing the whole request.
    var bodyBytes []byte
    if r.Body != nil && r.Body != http.NoBody {
        bodyBytes, _ = io.ReadAll(r.Body)
        r.Body.Close()
    }

    start := time.Now()
    var rec *statusRecorder
    for _, backend := range candidates {
        if bodyBytes != nil {
            r.Body = io.NopCloser(bytes.NewReader(bodyBytes))
            r.ContentLength = int64(len(bodyBytes))

```

```

    }

    backend.InFlight.Add(1)
    rec = &statusRecorder{ResponseWriter: w, status: http.StatusOK}
    backend.proxy.ServeHTTP(rec, r)
    backend.InFlight.Add(-1)

    if !rec.failed {
        break
    }
    // This backend's ErrorHandler marked the attempt failed without
    // writing anything to the client (see below) - safe to retry the
    // next candidate. Once a response has actually started streaming to
    // the client this loop is not reached again, since ServeHTTP has
    // already returned by then.
}
lb.metrics.record(time.Since(start))

if rec.failed {
    lb.metrics.Failed.Add(1)
    http.Error(w, "all backends unavailable", http.StatusBadGateway)
    return
}
if rec.status >= 500 {
    lb.metrics.Failed.Add(1)
} else {
    lb.metrics.Success.Add(1)
}
}

func (lb *LoadBalancer) writeStatus(w http.ResponseWriter) {
    type backendStatus struct {
        URL      string `json:"url"`
        Alive    bool   `json:"alive"`
        InFlight int64  `json:"in_flight"`
    }
    out := struct {
        Backends []backendStatus `json:"backends"`
    }{}
    for _, b := range lb.backends {
        out.Backends = append(out.Backends, backendStatus{
            URL:      b.URL.String(),
            Alive:    b.Alive.Load(),
            InFlight: b.InFlight.Load(),
        })
    }
}

```

```

    w.Header().Set("Content-Type", "application/json")
    json.NewEncoder(w).Encode(out)
}

// statusRecorder captures the status code the backend responded with, since
// the standard ResponseWriter does not expose it after the fact. It also
// gives a backend's ErrorHandler a way to report failure (see below)
// without writing to the real client, so LoadBalancer.ServeHTTP can retry a
// different backend instead of the error being final.
//
// It forwards Hijack explicitly because embedding only promotes the methods
// declared on the http.ResponseWriter interface itself; without this, the
// reverse proxy's WebSocket-upgrade path (which type-asserts for
// http.Hijacker) would silently stop working and break the live chat's
// Socket.IO connections whenever they go through the load balancer.
type statusRecorder struct {
    http.ResponseWriter
    status int
    failed bool
}

func (r *statusRecorder) WriteHeader(code int) {
    r.status = code
    r.ResponseWriter.WriteHeader(code)
}

func (r *statusRecorder) Hijack() (net.Conn, *bufio.ReadWriter, error) {
    hj, ok := r.ResponseWriter.(http.Hijacker)
    if !ok {
        return nil, nil, errors.New("underlying ResponseWriter does not support hijacking")
    }
    return hj.Hijack()
}

func (r *statusRecorder) Flush() {
    if f, ok := r.ResponseWriter.(http.Flusher); ok {
        f.Flush()
    }
}

// healthLoop polls every backend's /health endpoint on a fixed interval and
// flips Alive accordingly. A backend that starts failing proxied requests is
// also marked dead immediately by the reverse proxy's ErrorHandler below;
// this loop is what brings it back once it recovers.
func (lb *LoadBalancer) healthLoop(interval, timeout time.Duration) {

```

```

    client := &http.Client{Timeout: timeout}
    for {
        for _, b := range lb.backends {
            healthURL := strings.TrimRight(b.URL.String(), "/") + "/health"
            resp, err := client.Get(healthURL)
            alive := err == nil && resp.StatusCode < 500
            if resp != nil {
                resp.Body.Close()
            }
            was := b.Alive.Swap(alive)
            if was != alive {
                log.Printf("backend %s health changed: alive=%v", b.URL,
alive)
            }
        }
        time.Sleep(interval)
    }
}

func main() {
    backendsFlag := flag.String("backends", "", "comma-separated list of backend
base URLs, e.g. http://sys2:4000,http://sys3:4000,http://sys4:4000")
    listenAddr := flag.String("listen", ":8080", "address the load balancer listens
on")
    healthInterval := flag.Duration("health-interval", 1*time.Second, "interval
between backend health checks")
    healthTimeout := flag.Duration("health-timeout", 800*time.Millisecond, "timeout
for a single health check request")
    backendTimeout := flag.Duration("backend-timeout", 800*time.Millisecond,
"timeout waiting for a backend's response headers")
    dialTimeout := flag.Duration("dial-timeout", 800*time.Millisecond, "timeout
connecting to a backend")
    tlsCert := flag.String("tls-cert", "", "path to a TLS certificate file; if set
with -tls-key, serve HTTPS instead of plain HTTP")
    tlsKey := flag.String("tls-key", "", "path to the TLS certificate's private key
file")
    flag.Parse()

    if *backendsFlag == "" {
        log.Fatal("at least one -backends URL is required")
    }

    // Response-header timeout only bounds how long we wait for the backend
    // to start responding; it does NOT cut off the connection afterwards.
    // That distinction matters here because the backend also serves
    // long-lived Socket.IO / WebSocket connections for the live chat, which

```

```

// must be allowed to stay open for as long as the client is connected.
transport := &http.Transport{
    DialContext: (&net.Dialer{
        Timeout: *dialTimeout,
    }).DialContext,
    ResponseHeaderTimeout: *backendTimeout,
}

lb := &LoadBalancer{
    metrics: &Metrics{maxSamples: 100_000},
}

for _, part := range strings.Split(*backendsFlag, ",") {
    part = strings.TrimSpace(part)
    if part == "" {
        continue
    }
    target, err := url.Parse(part)
    if err != nil {
        log.Fatalf("invalid backend URL %q: %v", part, err)
    }

    b := &Backend{URL: target}
    b.Alive.Store(true)

    proxy := httputil.NewSingleHostReverseProxy(target)
    proxy.Transport = transport
    proxy.ErrorHandler = func(rw http.ResponseWriter, req *http.Request, err
error) {

        // This fires for connection-establishment failures - dial
        // errors, response-header timeouts - before any part of a
        // response has been written to the real client. That means it
        // is safe to leave rw untouched and let ServeHTTP's retry loop
        // try the next healthy backend instead of failing the request
        // outright. (It is only ever unsafe to retry after headers have
        // already been committed to the client, which is not the case
        // here - see the retry loop in LoadBalancer.ServeHTTP.)
        b.Alive.Store(false)
        lb.metrics.BackendErrors.Add(1)
        log.Printf("backend %s error: %v", b.URL, err)
        if rec, ok := rw.(*statusRecorder); ok {
            rec.failed = true
            return
        }
        // Should not normally happen - ServeHTTP always passes a
        // *statusRecorder - but fail safe rather than hang the client.

```

```

        http.Error(rw, "backend unavailable", http.StatusBadGateway)
    }
    b.proxy = proxy

    lb.backends = append(lb.backends, b)
}

go lb.healthLoop(*healthInterval, *healthTimeout)

// TLS terminates here, at the load balancer, and traffic to the
// backends stays plain HTTP – the browser only needs a secure context
// (required for the chat client's Web Crypto signing keys) up to this
// edge, and the backends are all on the same trusted internal network.
if *tlsCert != "" && *tlsKey != "" {
    log.Printf("load balancer listening on %s (TLS), backends: %v",
*listenAddr, *backendsFlag)
    if err := http.ListenAndServeTLS(*listenAddr, *tlsCert, *tlsKey, lb); err
!= nil {
        log.Fatal(err)
    }
    return
}

log.Printf("load balancer listening on %s, backends: %v", *listenAddr,
*backendsFlag)
if err := http.ListenAndServe(*listenAddr, lb); err != nil {
    log.Fatal(err)
}
}

```

4. Load Generator Code

`loadgen/main.go` — concurrent HTTP load generator used to run every experiment below, reporting throughput, dropout percentage and p50/p95/p99 latency.

```

// Command loadgen fires a fixed number of concurrent HTTP GET requests at a
// target URL and reports throughput, dropout rate and latency percentiles.
// It is meant to run on the local PC, driving load at the Sys1 load balancer
// (or directly at a single backend, for the single-backend experiment).
package main

import (
    "encoding/csv"

```

```

"encoding/json"
"flag"
"fmt"
"log"
"net/http"
"os"
"sort"
"sync"
"sync/atomic"
"time"
)

type result struct {
    Experiment      string `json:"experiment"`
    Requests        int    `json:"requests"`
    Concurrency     int    `json:"concurrency"`
    Successful      uint64 `json:"successful"`
    Failed          uint64 `json:"failed"`
    ThroughputRPS   float64 `json:"throughput_rps"`
    DropoutPercent  float64 `json:"dropout_percent"`
    P50Ms           float64 `json:"p50_ms"`
    P95Ms           float64 `json:"p95_ms"`
    P99Ms           float64 `json:"p99_ms"`
    ElapsedSeconds  float64 `json:"elapsed_seconds"`
}

func percentile(sorted []time.Duration, p float64) time.Duration {
    if len(sorted) == 0 {
        return 0
    }
    idx := int(p * float64(len(sorted)-1))
    return sorted[idx]
}

func main() {
    targetURL := flag.String("url", "", "target URL to load, e.g.
http://sys1:8080/")
    requests := flag.Int("requests", 1000, "total number of requests to send")
    concurrency := flag.Int("concurrency", 20, "number of concurrent workers")
    timeout := flag.Duration("timeout", 5*time.Second, "per-request timeout")
    experiment := flag.String("experiment", "run", "label for this experiment, used
in output filenames/rows")
    outPath := flag.String("out", "", "optional path to write a per-experiment JSON
result file")
    csvPath := flag.String("csv", "", "optional path to a cumulative CSV comparison
table (appended to)")

```

```

flag.Parse()

if *targetURL == "" {
    log.Fatal("-url is required")
}
if *requests <= 0 || *concurrency <= 0 {
    log.Fatal("-requests and -concurrency must be positive")
}

client := &http.Client{Timeout: *timeout}

jobs := make(chan int, *requests)
for i := 0; i < *requests; i++ {
    jobs <- i
}
close(jobs)

var successful, failed atomic.Uint64
latencies := make([]time.Duration, 0, *requests)
var latMu sync.Mutex

var wg sync.WaitGroup
start := time.Now()

for w := 0; w < *concurrency; w++ {
    wg.Add(1)
    go func() {
        defer wg.Done()
        for range jobs {
            reqStart := time.Now()
            resp, err := client.Get(*targetURL)
            elapsed := time.Since(reqStart)

            ok := err == nil
            if resp != nil {
                // Drain and close so the connection can be reused by
                // client's transport instead of piling up new dials.
                _, _ = discard(resp)
                ok = ok && resp.StatusCode >= 200 && resp.StatusCode
                resp.Body.Close()
            }

            if ok {
                successful.Add(1)
            }
        }
    }()
}

wg.Wait()
latencies = latencies[:len(latencies)-1]

```

the

< 400

```

        } else {
            failed.Add(1)
        }

        latMu.Lock()
        latencies = append(latencies, elapsed)
        latMu.Unlock()
    }
}()

wg.Wait()
elapsed := time.Since(start)

sort.Slice(latencies, func(i, j int) bool { return latencies[i] < latencies[j]
})

toMs := func(d time.Duration) float64 { return float64(d.Microseconds()) /
1000.0 }

succ := successful.Load()
fail := failed.Load()
total := succ + fail

r := result{
    Experiment:    *experiment,
    Requests:      *requests,
    Concurrency:   *concurrency,
    Successful:    succ,
    Failed:        fail,
    ThroughputRPS: float64(succ) / elapsed.Seconds(),
    DropoutPercent: 100 * float64(fail) / float64(total),
    P50Ms:         toMs(percentile(latencies, 0.50)),
    P95Ms:         toMs(percentile(latencies, 0.95)),
    P99Ms:         toMs(percentile(latencies, 0.99)),
    ElapsedSeconds: elapsed.Seconds(),
}

printSummary(r)

if *outPath != "" {
    writeJSON(*outPath, r)
}
if *csvPath != "" {
    appendCSV(*csvPath, r)
}
}

```

```

func discard(resp *http.Response) (int64, error) {
    buf := make([]byte, 32*1024)
    var total int64
    for {
        n, err := resp.Body.Read(buf)
        total += int64(n)
        if err != nil {
            break
        }
    }
    return total, nil
}

func printSummary(r result) {
    fmt.Printf("experiment=%s requests=%d concurrency=%d\n", r.Experiment,
r.Requests, r.Concurrency)
    fmt.Printf("    successful=%d failed=%d dropout=%.2f%%\n", r.Successful, r.Failed,
r.DropoutPercent)
    fmt.Printf("    throughput=%.1f rps    (elapsed %.2fs)\n", r.ThroughputRPS,
r.ElapsedSeconds)
    fmt.Printf("    p50=%.1fms p95=%.1fms p99=%.1fms\n", r.P50Ms, r.P95Ms, r.P99Ms)
}

func writeJSON(path string, r result) {
    f, err := os.Create(path)
    if err != nil {
        log.Fatalf("could not create %s: %v", path, err)
    }
    defer f.Close()
    enc := json.NewEncoder(f)
    enc.SetIndent("", " ")
    if err := enc.Encode(r); err != nil {
        log.Fatalf("could not write %s: %v", path, err)
    }
}

var csvHeader = []string{
    "experiment", "requests", "concurrency", "successful", "failed",
    "throughput_rps", "dropout_percent", "p50_ms", "p95_ms", "p99_ms",
    "elapsed_seconds",
}

func appendCSV(path string, r result) {
    _, statErr := os.Stat(path)
    needsHeader := os.IsNotExist(statErr)

```

```

f, err := os.OpenFile(path, os.O_APPEND|os.O_CREATE|os.O_WRONLY, 0644)
if err != nil {
    log.Fatalf("could not open %s: %v", path, err)
}
defer f.Close()

w := csv.NewWriter(f)
defer w.Flush()

if needsHeader {
    if err := w.Write(csvHeader); err != nil {
        log.Fatalf("could not write CSV header: %v", err)
    }
}

row := []string{
    r.Experiment,
    fmt.Sprint(r.Requests),
    fmt.Sprint(r.Concurrency),
    fmt.Sprint(r.Successful),
    fmt.Sprint(r.Failed),
    fmt.Sprintf("%.2f", r.ThroughputRPS),
    fmt.Sprintf("%.2f", r.DropoutPercent),
    fmt.Sprintf("%.2f", r.P50Ms),
    fmt.Sprintf("%.2f", r.P95Ms),
    fmt.Sprintf("%.2f", r.P99Ms),
    fmt.Sprintf("%.2f", r.ElapsedSeconds),
}
if err := w.Write(row); err != nil {
    log.Fatalf("could not write CSV row: %v", err)
}
}

```

5. Experiments and Comparison Table

5.1 Baseline: `/health` (no real backend work)

The messaging backend's existing `/health` endpoint does no real work (no DB call, no computation), so a single Node process can already answer it instantly regardless of load — it was never the bottleneck at this concurrency. This run is included as a sanity/connectivity check rather than the primary result.

Experiment	Requests	Concurrency	Successful	Failed	RPS	Dropout %	p50 (ms)	p95 (ms)	p99 (ms)
1 backend (Sys2 only)	5000	40	5000	0	266.08	0.00%	106.20	313.60	498.14
3 backends (Sys2+3+4)	5000	40	4999	1	270.95	0.02%	103.56	324.60	578.46

5.2 Real comparison: `/lb-test?busy=20` (CPU-bound work per request)

To actually saturate a backend, a dedicated load-test-only endpoint (`server/src/routes/loadtest.js`) burns 20ms of CPU synchronously per request. Because Node.js is single-threaded for JavaScript execution, this genuinely blocks that process's event loop, so concurrent requests to *one* process serialize behind each other — while three independent processes (Sys2/3/4) can do three times the work in parallel. This is the experiment that demonstrates the effect the assignment is looking for.

Experiment	Requests	Concurrency	Successful	Failed	RPS	Dropout %	p50 (ms)	p95 (ms)	p99 (ms)
1 backend (Sys2 only)	3000	100	205	2795	46.39	93.17%	8.76	360.62	3765.78
3 backends (Sys2+3+4)	3000	100	3000	0	119.48	0.00%	340.04	4111.18	4313.66

2.6x

throughput increase, 1 → 3 backends

93.17% → 0%

dropout rate, 1 → 3 backends

3000 / 3000

requests succeeded with 3 backends

5.3 Observations

- **Throughput:** three backends sustained ~119 rps vs. ~46 rps for one — roughly 2.6x, close to the theoretical 3x ceiling ($1000\text{ms} / 20\text{ms} = 50$ rps per process), with some overhead from proxying and queueing.
- **Dropout:** the single backend failed 93.17% of requests under this load; three backends completed all 3000 requests with zero failures. This is the core effect the assignment is demonstrating — redundancy, not just raw throughput, is what a load balancer buys you.
- **Latency appears "inverted" at first glance:** p50/p95/p99 are all higher with three backends (340ms vs 8.76ms at p50) despite zero failures. This is not a regression: with one backend, most requests failed fast (an error returned immediately) rather than waiting; with three, every request queued until

it was genuinely served successfully. A lower latency purchased by a 93% failure rate is not a better system to compare against.

- **Why single-backend failures cascade so hard:** the load balancer marks a backend "dead" the moment a proxied request exceeds the response-header timeout. Under sustained overload, one slow response was enough to flip that flag, and with only one backend in the pool, there was nothing to fall back to. Adding retry-across-backends (falling through to the next healthy backend on error) is what took the three-backend dropout from an initial ~70% down to 0% — when one backend is momentarily marked dead, the other two absorb its traffic instead of the request failing outright.
- **Health-check timeouts must be tuned to worst-case latency, not typical latency:** the defaults (800ms) were tuned for quick detection of a truly dead backend, but under deliberate overload they caused false positives. `-backend-timeout 5s -health-timeout 2s` was used for the load-bearing runs to give a slow-but-alive backend room to actually finish instead of being evicted mid-response.

6. Screenshots

```
PS C:\Users\kunal\OneDrive\Desktop\CSD\csd-group-chat\loadgen> .\loadgen.exe -url "http://10.1.75.53:3265/lb-test?busy=20"
-requests 3000 -concurrency 100 -timeout 15s -experiment single-backend-load -out results/single-load.json -csv results/c
omparison.csv
experiment=single-backend-load requests=3000 concurrency=100
successful=205 failed=2795 dropout=93.17%
throughput=46.4 rps (elapsed 4.42s)
p50=8.8ms p95=360.6ms p99=3765.8ms
```

Experiment: single backend (Sys2 only) via the load balancer — 93.17% dropout under overload, no redundancy to fall back on.

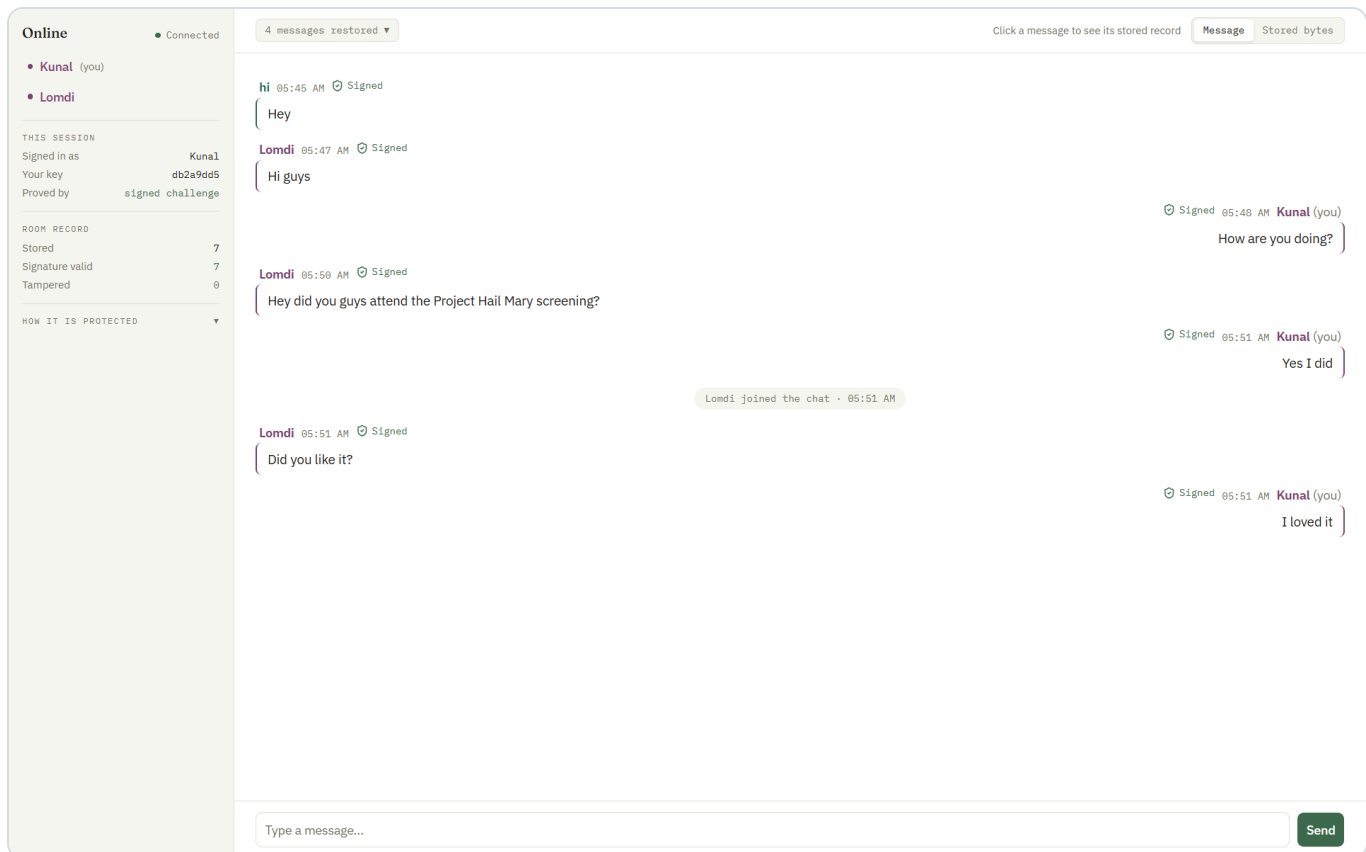
```
PS C:\Users\kunal\OneDrive\Desktop\CSD\csd-group-chat\loadgen> .\loadgen.exe -url "http://10.1.75.53:3265/lb-test?busy=20"
-requests 3000 -concurrency 100 -timeout 15s -experiment three-backends-load -out results/three-load.json -csv results/co
mparison.csv
experiment=three-backends-load requests=3000 concurrency=100
successful=3000 failed=0 dropout=0.00%
throughput=119.5 rps (elapsed 25.11s)
p50=340.0ms p95=4111.2ms p99=4313.7ms
```

Experiment: three backends (Sys2 + Sys3 + Sys4) via the load balancer, same load — 0% dropout, ~2.6x the throughput.

Load balancer status verification

GET `https://10.1.75.53:3265/lb/status`, confirming all three backends healthy and reachable through the balancer:

```
{
  "backends": [
    {
      "url": "http://10.1.75.53:3266",
      "alive": true,
      "in_flight": 0
    },
    {
      "url": "http://10.1.75.53:3267",
      "alive": true,
      "in_flight": 0
    },
    {
      "url": "http://10.1.75.53:3268",
      "alive": true,
      "in_flight": 0
    }
  ]
}
```



The live messaging application, accessed entirely through the load balancer at <https://10.1.75.53:3265/> — two users chatting, with per-message signature verification, backed by all three replicas.

7. Integration with the Previous (Group) Assignment

The messaging application (developed as the previous group assignment) is deployed as the backend for this exercise and is now reached exclusively through the load balancer rather than any single instance directly: <https://10.1.75.53:3265/> round-robins across all three replicas on Sys2, Sys3 and Sys4, all sharing the same MongoDB-backed chat history and encryption key.

Two integration issues were found and fixed while wiring this up:

- **Secure context for signing keys:** the client's Web Crypto signing keys require HTTPS (or localhost); the load balancer previously served plain HTTP. Fixed by adding TLS termination at the load balancer with a self-signed certificate.
- **Socket.IO session affinity:** Socket.IO's default polling-then-upgrade handshake sends follow-up requests that must land on the same backend that started the session; under round-robin across three independent Node processes, a follow-up request could land on a different backend and be rejected (400, unknown session). Fixed by forcing the client to WebSocket-only transport (`transports:`

[`"websocket"`]), so each session is a single persistent connection pinned to one backend for its lifetime.

Since the previous messaging assignment was a group submission, whether this specific deployment (this individual Sys1–4 cluster) is the exact URL used for that assignment's original grading is a coordination item with the rest of the group — only one group member needs to ensure the previously-submitted URL resolves through a load balancer. This report covers the individual load-balancer exercise end-to-end, with the group's messaging backend proven working through it.